MODULE 13

Claude Code Parallel Tasks & PR Workflows

Duration	Format	Audience	Day / Position
45 min	**Lecture + Demo**	**Mixed Engineers**	**Day 2 / Fifth**

LEARNING OBJECTIVES

By the end of this module, participants will be able to:

1. Use Claude Code background tasks (Ctrl+B, /tasks) to run long operations without blocking the main session
2. Set up git worktrees for parallel feature branches, enabling multiple Claude sessions on the same repository
3. Manage parallel development workflows: one branch per feature, independent Claude sessions, merge when ready
4. Generate professional PR descriptions using Claude Code with conventional commit formatting
5. Automate code review with Claude Code: reviewer subagent on every PR, structured feedback, review checklists
6. Design a parallel development strategy for TI projects: which tasks parallelize safely and which must be sequential

SESSION TIMING BREAKDOWN

0:00 – 0:08	**Section 1: Background Tasks & Long-Running Agents**
0:08 – 0:18	**Section 2: Git Worktrees for Parallel Claude Sessions**
0:18 – 0:28	**Section 3: PR Description Generation**
0:28 – 0:37	**Section 4: Automated Code Review Workflows**
0:37 – 0:43	**Section 5: Parallel Development Strategy**
0:43 – 0:45	**Wrap-Up: Quick Reference & Transition**

0:00 – 0:08	SECTION 1: Background Tasks & Long-Running Agents

1. Background Tasks & Long-Running Agents

1.1 Welcome to Day 2 Afternoon

This afternoon covers the advanced patterns that turn Claude Code from a personal tool into a team-scale development platform. Module 13 focuses on doing more at once: running tasks in the background, working on multiple features simultaneously, and automating PR workflows.

1.2 Background Tasks

Some Claude Code tasks take minutes: codebase analysis, large refactors, comprehensive test generation. Background tasks let you continue working while Claude runs these operations:

```
# During any Claude operation, press Ctrl+B to background it

# Example workflow:
> Generate comprehensive tests for all modules in src/
# Claude starts working... this will take 2-3 minutes
# Press Ctrl+B to send it to background

# Continue working on something else:
> Fix the import error in analyzers.py

# Check background task status:
> /tasks
# Shows: [1] test-generation (running, 2m 15s)

# When done, bring it back:
> /tasks  # Select task to see results
```

Foreground (Default)	Background (Ctrl+B)
Blocks your session until complete	You continue working immediately
See output in real-time	Results available when you check /tasks
Good for: quick edits, debugging, Q&A	Good for: test generation, codebase analysis, documentation
One task at a time	Multiple tasks running simultaneously

☆ **KEY POINT:** Background tasks are ideal for “fire and forget” operations: “Generate tests for all modules”, “Audit the codebase for security issues”, “Generate documentation for every public function.” These are tasks where you don’t need the results immediately.

1.3 Configuring Automatic Backgrounding

```
# Add to CLAUDE.md for automatic background rules:
## Background Execution Rules
Run in background automatically:
```

```
- Codebase exploration and analysis tasks
- Security audits and performance profiling
- Documentation generation across multiple files
- Any task where results aren't needed immediately

Never background:
- Code editing (need to see diffs immediately)
- Debugging (interactive process)
- File creation (need to verify output)
```

0:08 – 0:18	SECTION 2: Git Worktrees for Parallel Claude Sessions

2. Git Worktrees for Parallel Claude Sessions

2.1 The Problem: One Branch Per Terminal

Standard git allows only one branch checked out at a time per working directory. If you're working on feature-A and need to start feature-B, you have to stash, switch branches, and lose context. Git worktrees solve this by letting you check out multiple branches in separate directories simultaneously.

```
# Create worktrees for parallel development:
cd ~/my-sensor-toolkit

# Main branch stays in the original directory:
git branch  # * main

# Create a worktree for feature-A:
git worktree add ../sensor-toolkit-feature-a feature-a
# Creates ~/sensor-toolkit-feature-a with feature-a checked out

# Create a worktree for feature-B:
git worktree add ../sensor-toolkit-feature-b feature-b
# Creates ~/sensor-toolkit-feature-b with feature-b checked out

# Now you have 3 directories, each with its own branch:
# ~/my-sensor-toolkit         (main)
# ~/sensor-toolkit-feature-a  (feature-a)
# ~/sensor-toolkit-feature-b  (feature-b)
```

2.2 Parallel Claude Sessions

```
# Terminal 1: Work on feature-a
cd ~/sensor-toolkit-feature-a
claude
> Add humidity calibration offset to cleaners.py

# Terminal 2: Work on feature-b (simultaneously!)
cd ~/sensor-toolkit-feature-b
claude
> Add CSV export to analyzers.py

# Each Claude session has its own context, its own files,
# and its own git branch. No conflicts between sessions.
```

Pattern	Worktrees	Best For
Independent features	2-3 worktrees, each with its own Claude session	Features that don't touch the same files
Bug fix during feature	1 worktree for feature, main for hotfix	Urgent fixes without losing feature context
Experiment vs. stable	1 worktree for experiment, main stays clean	Trying risky refactors safely

Review while coding	1 worktree for PR review, 1 for development	Reviewing a colleague's PR without switching

⚠ **WARNING:** Parallel development works only when agents touch different files. If feature-A and feature-B both modify validators.py, you'll get merge conflicts. Plan your parallel work to minimize overlap. Use the localization techniques from Module 11 to check for conflicts before starting.

```
# Clean up worktrees when done:
cd ~/my-sensor-toolkit
git worktree remove ../sensor-toolkit-feature-a
git worktree remove ../sensor-toolkit-feature-b
# Branches still exist, only the extra directories are removed
```

0:18 – 0:28	SECTION 3: PR Description Generation

3. PR Description Generation

3.1 Conventional Commit PR Descriptions

A good PR description saves reviewers time, documents decisions, and serves as future reference. Claude Code can generate professional PRs from your git diff and commit history:

```
# Generate PR description from current branch:
> Generate a GitHub PR description for this branch.
> Include:
> - Title in conventional commit format (feat/fix/refactor)
> - Summary of changes (2-3 sentences)
> - Detailed list of files modified with explanations
> - Testing: what tests were added/modified, coverage
> - Breaking changes (if any)
> - Reviewer checklist (what to look for)
> Save to pr-description.md

# For headless/CI use:
git diff origin/main...HEAD | claude -p \
  "Generate a conventional commit PR description." \
  --output-format json > pr.json
```

3.2 PR Template with Claude Code

```
# Create a team PR template that Claude follows:
# .github/PULL_REQUEST_TEMPLATE.md
## Summary
<!-- What does this PR do? Why? -->

## Changes
<!-- List all modified files with brief explanations -->

## Testing
<!-- What tests were added? Coverage? -->

## Sensor Validation Impact
<!-- Does this change any validation ranges or logic? -->

## Breaking Changes
<!-- List any breaking changes. None if N/A -->

## Reviewer Checklist
- [ ] Code follows TI coding standards
- [ ] All tests pass
- [ ] No hardcoded sensor values
- [ ] Type hints and docstrings present

# Then tell Claude to fill it:
> Fill in the PR template at .github/PULL_REQUEST_TEMPLATE.md
> based on my current branch changes vs. main.
```

> ☆ **KEY POINT:** Add the PR template to your repository so every team member's PRs follow the same structure. Claude reads the template and fills each section based on the actual changes. The "Sensor Validation Impact" section is TI-specific — customize templates for your team's domain.

Track A (IT Engineers)	Track B (Process Engineers)
Create PRs with API endpoint documentation, schema changes, and migration notes	Create PRs with data pipeline changes, CSV format updates, and threshold modifications
Include deployment instructions and rollback procedures	Include sample data verification steps and validation range changes

0:28 – 0:37	SECTION 4: Automated Code Review Workflows

4. Automated Code Review Workflows

4.1 Review Subagent in CI

Combine what you learned in Modules 9 (CI/CD) and 10 (subagents) to create an automated review pipeline that runs on every PR:

```
# .github/workflows/ai-review.yml
name: AI Code Review
on: [pull_request]
jobs:
  review:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with: { fetch-depth: 0 }
      - run: npm install -g @anthropic-ai/claude-code
      - name: AI Review
        env:
          ANTHROPIC_API_KEY: ${{ secrets.ANTHROPIC_API_KEY }}
        run: |
          DIFF=$(git diff origin/main...HEAD)
          echo "$DIFF" | claude -p \
            "Review this PR diff. Check for:
             1. Bugs and logic errors
             2. Security vulnerabilities (hardcoded secrets)
             3. TI coding standard violations
             4. Missing tests for new functions
             5. Sensor validation range correctness
             Rate each finding: CRITICAL/MAJOR/MINOR.
             Output structured JSON." \
            --output-format json > review.json
      - uses: actions/github-script@v7
        with:
          script: |
            const fs = require('fs');
            const r = JSON.parse(fs.readFileSync('review.json'));
            await github.rest.issues.createComment({
              ...context.repo,
              issue_number: context.issue.number,
              body: `## 🤖 AI Code Review\n${r.result}`
            });
```

4.2 Multi-Reviewer Pattern

```
# Run multiple specialized reviews in parallel:
- name: Security Review
  run: |
    echo "$DIFF" | claude -p \
      "Security review: check for SQL injection, XSS, \
       hardcoded secrets, unsafe deserialization." \
      --output-format json > security.json
```

```
- name: Performance Review
  run: |
    echo "$DIFF" | claude -p \
      "Performance review: check for O(n^2) loops, \
       unnecessary memory copies, missing caching." \
      --output-format json > perf.json

- name: Validation Review
  run: |
    echo "$DIFF" | claude -p \
      "Sensor validation review: check that ranges \
       match TI specs (temp -40..150, pres 0..1000)." \
      --output-format json > validation.json
```

☆ **KEY POINT:** The multi-reviewer pattern mirrors how TI teams actually review code: one person checks security, another checks performance, a domain expert checks sensor logic. With Claude, these reviews happen in parallel, complete in seconds, and provide a structured first pass before human reviewers focus on architecture and design.

0:37 – 0:43	SECTION 5: Parallel Development Strategy

5. Parallel Development Strategy

5.1 What Parallelizes Safely?

Task Pair	Parallelize?	Why
Add new module + Write tests for existing module	☑ Yes	Different files, no overlap
Refactor validators.py + Add new validation rules	✕ No	Same file — merge conflict guaranteed
Write docs + Run security audit	☑ Yes	Docs are new files, audit is read-only
Feature-A in module A + Feature-B in module B	☑ Yes	Independent modules
Refactor imports + Add new dependency	✕ No	Both touch import structure
Generate tests + Fix bug in tested function	⚠ Maybe	Tests may need to change if bug fix changes behavior

5.2 The Parallel Sprint Pattern

```
# Team parallel sprint using worktrees:

# 1. Plan: identify independent tasks
> Use the sensor-planner subagent to identify which
> of these 4 tasks can be developed in parallel:
> - Add humidity_offset to cleaners.py
> - Add csv_export to analyzers.py
> - Add API endpoint for validation
> - Write comprehensive tests for validators.py

# 2. Create worktrees for parallel tasks
git worktree add ../st-humidity feat/humidity-offset
git worktree add ../st-csv-export feat/csv-export

# 3. Work in parallel (two terminals)
# Terminal 1: cd ../st-humidity && claude
# Terminal 2: cd ../st-csv-export && claude

# 4. Merge back (one at a time)
cd ~/my-sensor-toolkit
git merge feat/humidity-offset
git merge feat/csv-export  # Resolve any conflicts

# 5. Clean up
git worktree remove ../st-humidity
git worktree remove ../st-csv-export
```

> ☆ **KEY POINT:** The key rule: parallel only works when tasks touch different files. Before starting parallel work, always run file localization (Module 11) to check for overlap. If two tasks both modify the same file, do them sequentially.

0:43 – 0:45	WRAP-UP: Quick Reference & Transition

6. Quick Reference & Transition

MODULE 13 RECAP

✅ Background tasks: Ctrl+B to background, /tasks to monitor. Ideal for test generation, audits, documentation.

✅ Git worktrees: git worktree add for parallel branches. Each worktree gets its own Claude session.

✅ Parallel sessions: multiple terminals, independent contexts, merge when ready. Only works with different files.

✅ PR descriptions: Claude fills PR templates from git diff. Conventional commit format. Automate in CI.

✅ Automated review: reviewer subagent on every PR via GitHub Actions. Structured feedback posted as PR comment.

✅ Multi-reviewer: parallel security + performance + validation reviews in CI. First-pass before human reviewers.

✅ Parallel strategy: localize files first, parallelize independent tasks, serialize overlapping ones.

QUICK REFERENCE CARD

Task	Command	Notes
Background task	Ctrl+B during operation	Check with /tasks
Create worktree	git worktree add ../dir branch-name	Separate directory per branch
Remove worktree	git worktree remove ../dir	Branch stays, dir removed
Generate PR desc	> Generate PR description from branch diff	Save to pr-description.md
PR desc in CI	git diff | claude -p "Generate PR desc" --output-format json	Post as PR comment
AI review in CI	git diff | claude -p "Review for bugs, security"	Run on pull_request
Multi-review	Parallel jobs: security, performance, validation	One CI workflow
Check overlap	Grep/Glob for shared files before parallelizing	Prevent merge conflicts

TRANSITION TO MODULE 14

Module 14 (Production Patterns & Technical Debt) covers the patterns that make AI-generated code production-ready: error handling, performance optimization, code quality metrics, dependency management, and systematic technical debt analysis with Claude Code.